

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I Dheeraj Krushna Pothala / 192525230 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P. DheerajKrushna

Q1. Single student-course table — normalization to 3NF (BL4)

Scenario: A university stores all data in one flat table.

```
StudentCourse (StudentID, StudentName, StudentCity,  
               CourseID, CourseTitle, Instructor, InstructorPhone, Grade)
```

Anomalies in the unnormalized table:

- Update anomaly: if an Instructor's phone changes, every row for every course they teach must be updated.
- Insert anomaly: a new course cannot be added until at least one student enrolls in it.
- Delete anomaly: deleting the last student enrolled in a course erases all record of that course and its instructor.

Step 1 — 1NF: all attributes are already atomic (no repeating groups), so the table is in 1NF.

Step 2 — 2NF: the key is (StudentID, CourseID). StudentName and StudentCity depend only on StudentID; CourseTitle, Instructor, InstructorPhone depend only on CourseID — these are partial dependencies on a composite key, so the table violates 2NF. Split off the partial dependencies:

```
Student(StudentID, StudentName, StudentCity)  
Course(CourseID, CourseTitle, Instructor, InstructorPhone)  
Enrollment(StudentID, CourseID, Grade)
```

Step 3 — 3NF: in Course, InstructorPhone depends on Instructor, not directly on CourseID (a transitive dependency CourseID -> Instructor -> InstructorPhone). Remove it:

```
Student(StudentID, StudentName, StudentCity)  
Course(CourseID, CourseTitle, InstructorID)  
Instructor(InstructorID, InstructorName, InstructorPhone)  
Enrollment(StudentID, CourseID, Grade)
```

This design is in 3NF: every non-key attribute depends on the whole key and nothing but the key, eliminating all three anomalies above.

Q2. Employees working in multiple departments (BL6)

Schema:

```
Employee(EmpID, EmpName)  
Department(DeptID, DeptName)  
EmpDept(EmpID, DeptID)  -- one row per employee-department assignment
```

Query — employees assigned to more than one department:

```
SELECT E.EmpID, E.EmpName, COUNT(DISTINCT ED.DeptID) AS NumDepartments  
FROM Employee E  
JOIN EmpDept ED ON E.EmpID = ED.EmpID  
GROUP BY E.EmpID, E.EmpName  
HAVING COUNT(DISTINCT ED.DeptID) > 1;
```

Query — names of the departments each such employee works in:

```
SELECT E.EmpName, D.DeptName  
FROM Employee E  
JOIN EmpDept ED ON E.EmpID = ED.EmpID
```

```

JOIN Department D ON ED.DeptID = D.DeptID
WHERE E.EmpID IN (
    SELECT EmpID FROM EmpDept GROUP BY EmpID HAVING COUNT(DISTINCT DeptID) > 1
)
ORDER BY E.EmpName;

```

Justification: a JOIN across Employee and EmpDept is needed because the count is per employee across many rows, so GROUP BY + HAVING is the natural aggregate filter. The junction table EmpDept models the many-to-many employee-department relationship correctly (an employee row is never duplicated with repeating department columns), and the subquery in the second query reuses the same HAVING logic to restrict which employees' department names get listed.

Q3. Functional dependencies — candidate keys and BCNF (BL5)

Given relation R(A, B, C, D, E) with functional dependencies:

```

A -> B, C
C -> D
B -> E

```

Candidate key: the closure of {A} is {A, B, C, D, E} = R, so A alone determines every attribute — A is a candidate key. No proper subset of any other combination produces a smaller key that also covers all attributes without A, so A is the only candidate key.

BCNF check: BCNF requires that for every non-trivial FD $X \rightarrow Y$, X must be a superkey.

- $A \rightarrow B, C$: A is the candidate key (superkey) — satisfies BCNF.
- $C \rightarrow D$: C is not a superkey (closure of {C} is only {C, D}) — violates BCNF.
- $B \rightarrow E$: B is not a superkey (closure of {B} is only {B, E}) — violates BCNF.

Since $C \rightarrow D$ and $B \rightarrow E$ violate BCNF, R must be decomposed.

Decomposition:

```

R1(C, D)      -- from C -> D
R2(B, E)      -- from B -> E
R3(A, B, C)   -- remaining attributes with key A

```

Verification: R3 has FD $A \rightarrow B, C$ where A is the key — BCNF holds. R1 has $C \rightarrow D$ where C is the key — BCNF holds. R2 has $B \rightarrow E$ where B is the key — BCNF holds. All three resulting relations are in BCNF, and R3 preserves the ability to reconstruct the original relation via natural joins on B and C (lossless decomposition), since A is a key of R3.

Q4. Banking system — views and constraints for security (BL6)

Schema:

```

Customer(CustID, CName, City, Phone)
Account(AccNo, CustID, AccType, Balance, BranchName)
Transaction(TxnID, AccNo, TxnType, Amount, TxnDate)

```

Integrity constraints:

```

ALTER TABLE Account ADD CONSTRAINT chk_balance CHECK (Balance >= 0);
ALTER TABLE Account ADD CONSTRAINT fk_acc_cust FOREIGN KEY (CustID) REFERENCES
Customer(CustID);
ALTER TABLE Transaction ADD CONSTRAINT fk_txn_acc FOREIGN KEY (AccNo) REFERENCES
Account(AccNo);
ALTER TABLE Transaction ADD CONSTRAINT chk_amount CHECK (Amount > 0);
ALTER TABLE Transaction ADD CONSTRAINT chk_txn_type CHECK (TxnType IN
('Deposit','Withdrawal'));

```

Views for different user types:

```

-- Teller: account summary only, no full transaction history
CREATE VIEW TellerAccountView AS
SELECT A.AccNo, C.CName, A.AccType, A.Balance, A.BranchName
FROM Account A JOIN Customer C ON A.CustID = C.CustID;

-- Customer self-service: only their own account's transactions (app filters by
CustID)
CREATE VIEW CustomerTxnView AS
SELECT A.CustID, A.AccNo, T.TxnID, T.TxnType, T.Amount, T.TxnDate
FROM Account A JOIN Transaction T ON A.AccNo = T.AccNo;

-- Branch manager: aggregated balances per branch, no individual customer data
CREATE VIEW BranchSummaryView AS
SELECT BranchName, COUNT(*) AS NumAccounts, SUM(Balance) AS TotalBalance
FROM Account
GROUP BY BranchName;

```

Justification: CHECK constraints prevent a negative balance and invalid transaction amounts/types at the database level, so integrity does not depend on application code being correct. Foreign keys stop orphaned accounts or transactions. TellerAccountView hides customer Phone; CustomerTxnView is granted only with a row-level filter on CustID so one customer cannot see another's transactions; BranchSummaryView exposes only aggregates to a manager role, never individual customer identities — each view grants exactly the columns/rows a role needs (least privilege), matching the security requirement.

Q5. Sales database anomalies — normalization to 3NF/BCNF (BL4)

Unnormalized relation:

```

Sales(OrderID, CustID, CustName, CustCity, ProductID, ProductName, UnitPrice, Qty,
SalesRepID, SalesRepName)

```

Redundancy/anomalies: CustName and CustCity repeat on every order the same customer places; ProductName and UnitPrice repeat on every order line for the same product; SalesRepName repeats for every order handled by the same rep. Deleting the only order for a product loses the product's name and price entirely (delete anomaly); updating a product's price means updating every historical order row (update anomaly).

Normalized decomposition (3NF):

```

Customer(CustID, CustName, CustCity)
Product(ProductID, ProductName, UnitPrice)

```

```
SalesRep(SalesRepID, SalesRepName)
Orders(OrderID, CustID, SalesRepID, OrderDate)
OrderLine(OrderID, ProductID, Qty)
```

Each table's non-key attributes now depend only on that table's own key: CustName/CustCity depend only on CustID, ProductName/UnitPrice only on ProductID, SalesRepName only on SalesRepID, and Qty depends on the composite (OrderID, ProductID). No transitive or partial dependencies remain, and every determinant (CustID, ProductID, SalesRepID, the OrderLine composite key) is a candidate key of its own table, so the design also satisfies BCNF.

Q6. Customers and orders — joins and subqueries for frequent customers (BL3)

Schema:

```
Customer(CustID, CName, City)
Orders(OrderID, CustID, OrderDate, TotalAmount)
```

Query using JOIN — customers and how many orders each has placed:

```
SELECT C.CustID, C.CName, COUNT(O.OrderID) AS NumOrders
FROM Customer C JOIN Orders O ON C.CustID = O.CustID
GROUP BY C.CustID, C.CName
ORDER BY NumOrders DESC;
```

Query using subquery — "frequent customers": those who have placed more orders than the average customer:

```
SELECT CName
FROM Customer
WHERE CustID IN (
    SELECT CustID FROM Orders
    GROUP BY CustID
    HAVING COUNT(*) > (
        SELECT AVG(cnt) FROM (
            SELECT COUNT(*) AS cnt FROM Orders GROUP BY CustID
        ) AS PerCustomer
    )
);
```

Query using subquery — customers who placed an order above the overall average order amount:

```
SELECT DISTINCT C.CName
FROM Customer C
JOIN Orders O ON C.CustID = O.CustID
WHERE O.TotalAmount > (SELECT AVG(TotalAmount) FROM Orders);
```

Logic: the JOIN attaches each order to its customer for a direct count. The nested subquery computes the system-wide average order count per customer and then filters customers whose own order count exceeds it — this comparison against an aggregate-of-aggregates is naturally expressed as a subquery rather than a join, since it needs a single scalar reference value computed independently of any one customer's rows.

Q7. Multi-valued dependency — decomposition to 4NF (BL4)

Given relation Faculty(FacultyID, Subject, Hobby) where a faculty member can teach several subjects and independently pursue several hobbies, with Subject and Hobby unrelated to each other:

```
Faculty(FacultyID, Subject, Hobby)
```

Sample rows:

```
F1, DBMS, Chess
F1, DBMS, Painting
F1, OS, Chess
F1, OS, Painting
```

Analysis: FacultyID $\twoheadrightarrow$ Subject and FacultyID $\twoheadrightarrow$ Hobby are multi-valued dependencies, because for a fixed FacultyID the set of Subjects is independent of the set of Hobbies — every combination of a faculty's subjects and hobbies must appear (as the sample rows show), causing redundancy (2 subjects x 2 hobbies = 4 rows to represent 2 independent facts). This relation is in BCNF (its only key is the full row) but not 4NF, because it has a non-trivial MVD that is not a functional dependency.

Decomposition into 4NF — split the two independent multi-valued facts into separate relations:

```
FacultySubject(FacultyID, Subject)
FacultyHobby(FacultyID, Hobby)
```

Each resulting relation has at most one multi-valued fact per FacultyID, so it satisfies 4NF, and the original relation can be reconstructed losslessly by taking the natural join of FacultySubject and FacultyHobby on FacultyID (the join reintroduces the same combinatorial rows, but the base tables themselves store each fact only once).

Q8. Improper constraints — integrity evaluation (BL5)

Problematic schema (as originally designed):

```
CREATE TABLE Account (
  AccNo VARCHAR(10),
  CustID VARCHAR(10),
  Balance DECIMAL(10,2),
  AccType VARCHAR(20)
);
```

Issues identified:

- No PRIMARY KEY on AccNo, so duplicate account numbers can be inserted.
- No FOREIGN KEY on CustID, so an account can reference a customer that does not exist (orphaned record).
- No CHECK constraint, so Balance can go negative or AccType can hold a meaningless value.
- No NOT NULL on required columns, allowing an account with no CustID or Balance.
- No UNIQUE / DEFAULT clauses, so accidental duplicates and missing default states (e.g., a new account defaulting to 0 balance) are not enforced.

Corrected schema:

```
CREATE TABLE Account (
  AccNo VARCHAR(10) PRIMARY KEY,
```

```
CustID    VARCHAR(10) NOT NULL REFERENCES Customer(CustID),
Balance   DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (Balance >= 0),
AccType   VARCHAR(20) NOT NULL CHECK (AccType IN ('Savings','Current'))
);
```

Evaluation: the corrected schema enforces entity integrity (PRIMARY KEY prevents duplicate/NULL account numbers), referential integrity (FOREIGN KEY prevents orphaned accounts), and domain integrity (CHECK and NOT NULL prevent invalid balances or account types) directly at the database layer, so inconsistent data cannot enter regardless of which application or script writes to the table — this is more reliable than relying on application-level validation alone.

Q9. Join dependency — decomposition to 5NF (BL6)

Scenario: a relation records which Supplier supplies which Part for which Project:

```
SupplyPPJ(Supplier, Part, Project)
```

If the business rule is "a supplier supplies a part, that part is used in a project, and that supplier supplies that project" — i.e. the three pairwise relationships (Supplier-Part, Part-Project, Supplier-Project) determine the valid combinations — then SupplyPPJ satisfies a join dependency JD(R1, R2, R3) but has no simpler MVD, so decomposing on any single pair alone would lose information; the relation must be split into all three pairwise relations and reconstructed by joining all three.

Decomposition into 5NF:

```
SupplierPart(Supplier, Part)
PartProject(Part, Project)
SupplierProject(Supplier, Project)
```

Reconstruction:

```
SELECT SP.Supplier, SP.Part, PP.Project
FROM SupplierPart SP
JOIN PartProject PP ON SP.Part = PP.Part
JOIN SupplierProject SPJ ON SP.Supplier = SPJ.Supplier AND PP.Project =
SPJ.Project;
```

Each of the three decomposed relations stores only a single pairwise fact, removing the redundancy of repeating whole (Supplier, Part, Project) triples whenever any one relationship is shared across multiple rows; the original data is recoverable only by joining all three tables together (a three-way join dependency), which is the defining property of 5NF.

Q10. Optimizing inefficient SQL queries (BL5)

Inefficient query: report the number of orders per customer using a correlated subquery evaluated once per customer row:

```
-- INEFFICIENT
SELECT C.CName,
       (SELECT COUNT(*) FROM Orders O WHERE O.CustID = C.CustID) AS OrderCount
FROM Customer C;
```

Optimized using JOIN + GROUP BY:

```
-- OPTIMIZED
SELECT C.CName, COUNT(O.OrderID) AS OrderCount
```



```
FROM Customer C
LEFT JOIN Orders O ON C.CustID = O.CustID
GROUP BY C.CustID, C.CName;
```

Further optimization — indexing the join column:

```
CREATE INDEX idx_orders_custid ON Orders(CustID);
```

Further optimization — a reusable view for repeated reporting:

```
CREATE VIEW CustomerOrderCount AS
SELECT C.CustID, C.CName, COUNT(O.OrderID) AS OrderCount
FROM Customer C LEFT JOIN Orders O ON C.CustID = O.CustID
GROUP BY C.CustID, C.CName;
```

Evaluation: the correlated subquery re-scans Orders once per customer ($O(n \times m)$ in the worst case), while the LEFT JOIN + GROUP BY scans each table once and lets the optimizer pick a single hash/merge join plan (roughly $O(n + m)$). LEFT JOIN is kept (not INNER JOIN) so customers with zero orders still appear with OrderCount = 0, preserving the same result set as the original query. The index on Orders.CustID lets the join use an index seek instead of a full table scan as Orders grows, and wrapping the pattern in a view avoids re-writing the join in every report that needs this figure.